Madhuben & Bhanubhai Patel Institute of Technology

(MBIT)

The Charutar Vidya Mandal(CVM) University

New V.V. Nagar

Mini Project Report

On

CareBuddy: An Interactive Elderly Safety and Reminder Assistant Using ESP32 and Smartphone Connectivity

Submitted by:

Md Shabbir Ahmad-12302040701108

Meet Chauhan-12302040701109

Mauneshkumar Shah-12302040701107

Under the guidance of: Dr. Kishori Shekokar

Abstract

As populations age, maintaining the independence and health of elderly people living at home has become a critical challenge. Many older adults find it hard to keep up with daily routines, medication schedules and hydration habits. Smartphone reminder applications exist, but their interfaces are often too complex, unintuitive or easy to ignore, while dedicated assistive medical devices can be expensive and lack engaging, approachable interaction.

This project presents **CareBuddy**, a low-cost interactive assistive device designed for elderly users. It combines an **ESP32 microcontroller**, an **SSD1306 OLED display**, a **TTP223 capacitive touch sensor** and an **active buzzer**. Animated eyes on the OLED convey expressions such as blinking, sleeping, happiness and reminder prompts, giving the device a companion-like presence, while an automatic sleep/idle state machine avoids unnecessary display activity. The device talks to a companion Android application over **Bluetooth Low Energy (BLE)** using a Nordic UART-style JSON message model. Caregivers configure reminders (medicine, hydration, meals) and emergency contacts in the app; when a reminder fires, CareBuddy alerts the user visually and audibly and accepts a simple touch as acknowledgment.

CareBuddy also provides an **emergency SOS workflow**: a deliberate long hold starts an on-screen countdown that lets the user cancel a false trigger before an SOS event is sent to the paired smartphone, which then runs the configured emergency communication. Functional testing of the firmware states, BLE connectivity and hardware interaction loops shows that affordable embedded hardware can deliver reliable reminders, intuitive tactile feedback and accessible emergency connectivity.

Table of Contents

Abstract 1
1. Introduction 3
    1.1 Project Summary and Introduction 3
    1.2 Problem Statement 3
    1.3 Aim and Objectives 4
    1.4 Scope of the Report 4
2. Existing Work and System 5
3. Software and Hardware Requirements 6
    3.1 ESP32 Microcontroller 6
    3.2 0.96-inch OLED SSD1306 Display 6
    3.3 TTP223 Capacitive Touch Sensor 6
    3.4 Active Buzzer Module 6
    3.5 Smartphone Application and BLE Connectivity 6
    3.6 Requirement Summary 6
4. System Overview 7
    4.1 System Architecture 7
    4.2 Hardware Wiring and Pin Configuration 7
    4.3 Firmware State Machines 8
    4.4 BLE Communication Protocol and JSON Messaging 9
    4.5 Emergency SOS Workflow 9
    4.6 Data Flow and Reminder Workflow 10
5. Results and System Behavior 11
    5.1 Experimental Setup and Prototyping 11
    5.2 Functional Testing and Verification 11
    5.3 Observed Prototype Behavior 11
    5.4 Limitations of the Current Prototype 12
6. Societal Relevance 13
7. Conclusion 14
8. Future Scope 15
9. References 16

1. Introduction

1.1 Project Summary and Introduction

CareBuddy is an interactive, low-cost assistive technology prototype designed to support elderly people with daily routines, medication adherence and emergency safety. By combining embedded hardware with smartphone connectivity, it provides a tangible, approachable companion device that sits in the living space or on the bedside table of an elderly user.

The project centres on an ESP32 microcontroller interfaced with an OLED screen that shows dynamic animated eyes, a capacitive touch sensor for simple interaction, a buzzer for audible alerts, and Bluetooth Low Energy (BLE) for synchronisation with a companion Android application. Rather than relying only on abstract smartphone notifications or complex smart-home ecosystems, CareBuddy offers an embodied, friendly interface that combines visual, auditory and tactile feedback.

1.2 Problem Statement

Ageing populations face growing challenges in independent living, routine management and timely medication intake. Cognitive decline, forgetfulness and unfamiliarity with modern smartphone interfaces often lead to missed medication and delayed responses to daily needs. The key challenges in elderly assistive care are:

- **Interface complexity:** smartphones and smart displays have crowded screens, small text and complex menus that frustrate or confuse elderly users.
- **Notification fatigue:** standard alarms and phone chimes are frequently ignored, misunderstood or hard to turn off.
- **Emergency response delays:** traditional medical alert systems often need subscription services, wearable pendants that users forget to wear, or multi-step dialling during moments of panic.
- **Lack of companion engagement:** static reminder devices fail to create an approachable human–device interaction, which leads to abandonment.

There is a clear need for an affordable, easy-to-use and engaging device that combines local tactile interaction, clear visual expression, reliable audible reminders and a seamless smartphone bridge for caregivers, without high cost or a steep learning curve.

1.3 Aim and Objectives

The aim of this project is to design, develop and test CareBuddy as an affordable, user-friendly assistant. The specific objectives are:

- Design a low-cost hardware prototype from commodity components centred on the ESP32 microcontroller.
- Implement human-centric visual and auditory feedback through animated eyes on an OLED display and configured buzzer patterns.
- Provide intuitive tactile interaction through a TTP223 capacitive touch sensor, supported by software debouncing and state-machine-based event detection.
- Establish smartphone integration over BLE with structured JSON messaging for reminders and configuration.
- Deliver an accessible emergency SOS mechanism with an interactive on-screen countdown that prevents false activations.

1.4 Scope of the Report

This report documents the architecture, hardware specification, firmware state machines, communication protocol, functional test results, societal relevance and future expansion of the CareBuddy project.

2. Existing Work and System

Existing solutions in elderly care range from conventional smartphone reminder applications to dedicated medical alert pendants and high-end smart-home ecosystems. Generic medication-reminder apps depend entirely on the user noticing, unlocking and navigating a phone screen, which is a real hurdle for people with visual impairment or cognitive fatigue.

Dedicated personal emergency response systems (PERS) offer physical panic buttons, but they are typically proprietary, expensive and subscription-based, and they lack everyday utility such as hydration or meal reminders, so they are often left in a drawer. Commercial smart displays such as the Amazon Echo Show or Google Nest Hub offer voice interaction and reminders, but they need constant internet connectivity, Wi-Fi provisioning and privacy-sensitive cloud accounts, and they cost more.

Table 1: Comparison of existing solutions with CareBuddy

Property	Phone reminder app	PERS pendant	Smart display	CareBuddy
Simple physical interaction	No	Yes (panic only)	Voice / touch	Yes
Daily reminders	Yes	No	Yes	Yes
Emergency SOS	Manual	Yes	Limited	Yes, with cancel
Works without internet	Yes	Varies	No	Yes (offline-first)
Subscription or cloud account	No	Often	Yes	No
Relative cost	Low	Medium to high	Higher	Low

CareBuddy addresses these gaps by providing three things. First, a **dedicated, distraction-free physical companion**: a standalone desktop device that does not require the elderly user to operate a smartphone or touchscreen. Second, **embodied expressive interaction**: animated eyes that establish a friendly presence and turn a utility reminder tool into an approachable companion. Third, a **transparent hybrid architecture**: local offline reminder execution backed by BLE smartphone synchronisation, so the device keeps working when internet connectivity is unstable or absent.

3. Software and Hardware Requirements

3.1 ESP32 Microcontroller

The ESP32 is a low-cost, low-power system-on-a-chip with integrated Wi-Fi and dual-mode Bluetooth (Classic and BLE). In CareBuddy it runs the firmware loop, stores reminders in non-volatile memory (Preferences/NVS), drives the display and sensors, and maintains the BLE link to the Android app.

3.2 0.96-inch OLED SSD1306 Display

A compact I2C monochrome OLED (128 × 64 pixels) renders visual feedback, system status messages and the animated eyes. Its high contrast stays visible in varying light.

3.3 TTP223 Capacitive Touch Sensor

The TTP223 is a single-channel capacitive touch detector with a digital output. Simple touches wake the device, acknowledge reminders, check status or start an SOS.

3.4 Active Buzzer Module

An active buzzer gives distinct tones for reminders, acknowledgment and emergency countdown warnings.

3.5 Smartphone Application and BLE Connectivity

A companion Android application written in Kotlin is the caregiver interface. It lets the caregiver set reminders and emergency contacts and syncs them to the ESP32 over BLE.

3.6 Requirement Summary

Table 2: Hardware and software components

Component / Tool	Specification / Role
ESP32 NodeMCU module	Central microcontroller for firmware logic, peripherals and BLE communication
0.96-inch OLED display	I2C interface (SSD1306 driver), 128 × 64 resolution; renders animated eyes and text alerts
TTP223 touch sensor	Capacitive module for user input (short touch, long press, SOS hold)
Active buzzer	Audible feedback, reminder chimes and emergency alert patterns
Arduino IDE / C++	Firmware development environment and language
Android Studio / Kotlin	Companion mobile application development

4. System Overview

4.1 System Architecture

CareBuddy follows a modular client–server style architecture. The smartphone application manages configuration data and user preferences and communicates over BLE with the standalone ESP32 companion device, which executes reminders locally and drives the OLED, touch sensor and buzzer (Figure 1).

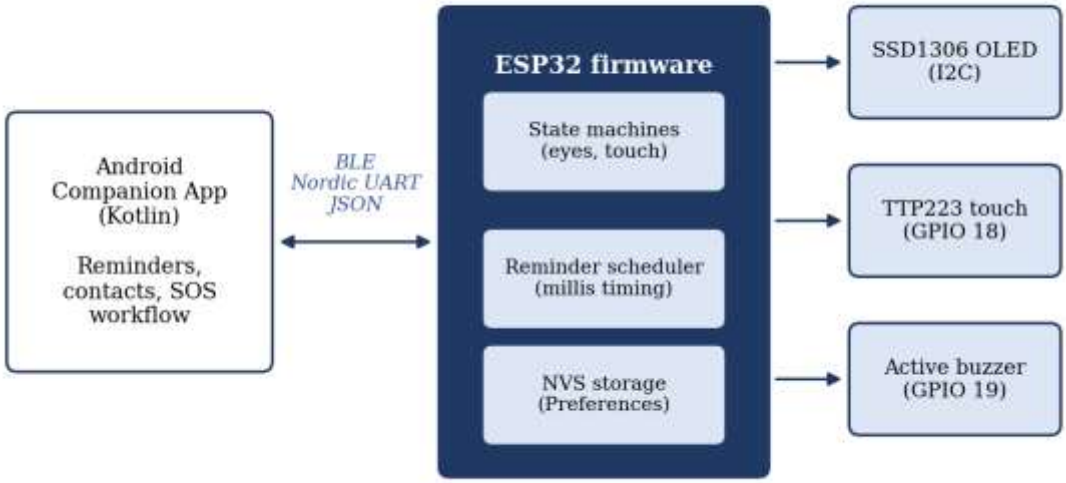


Figure 1: CareBuddy system architecture

4.2 Hardware Wiring and Pin Configuration

The prototype connects its peripherals to specific GPIO pins of the ESP32 development board, as listed in Table 3 and shown in Figure 2.

Table 3: Pin configuration

Peripheral	Component pin	ESP32 GPIO	Description
OLED display	VCC	3.3V	Power supply
OLED display	GND	GND	Ground
OLED display	SDA	GPIO 21 (D21)	I2C data line
OLED display	SCL	GPIO 22 (D22)	I2C clock line
TTP223 touch	OUT	GPIO 18 (D18)	Digital touch output
Buzzer	Positive (+)	GPIO 19 (D19)	Digital / PWM output control

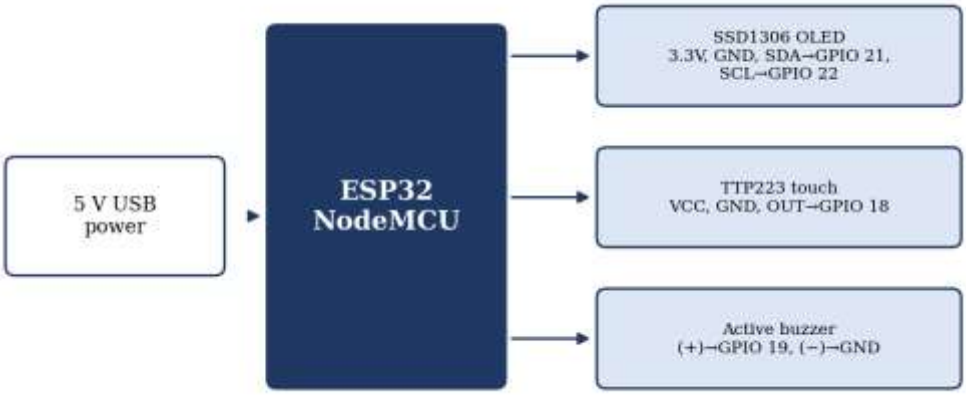


Figure 2: Wiring block diagram of the prototype

4.3 Firmware State Machines

To manage behavioural transitions cleanly without blocking execution, the ESP32 firmware uses deterministic state machines for both the visual expressions and the touch-event detection.

Eye expression states are STARTUP, IDLE, BLINKING, LOOK_LEFT, LOOK_RIGHT, HAPPY, CONFUSED, IRRITATED, SLEEPY, SLEEPING, WAKING, REMINDER, STATUS, SOS_COUNTDOWN and SOS_SENT. After inactivity the device drifts from IDLE to SLEEPY and SLEEPING, which minimises display activity, and returns through WAKING to IDLE on a valid touch (Figure 3).

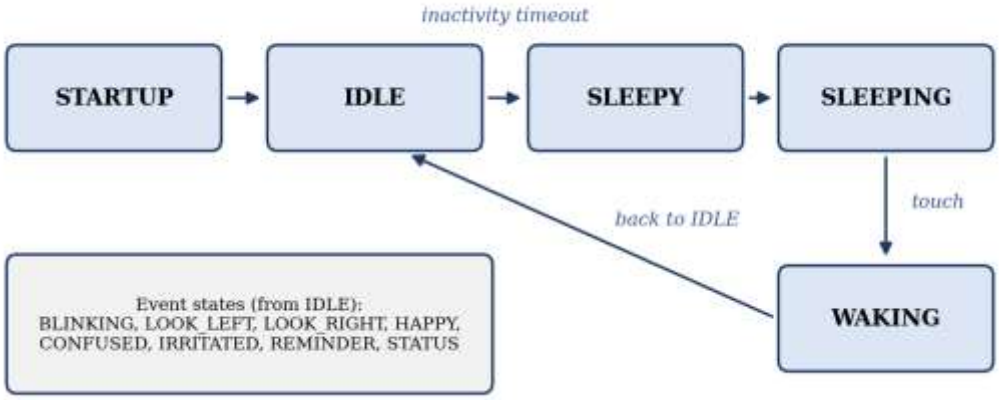


Figure 3: Eye-expression lifecycle and sleep/wake behaviour

Touch detection states are IDLE, DEBOUNCING, PRESSED, WAIT_RELEASE, WAIT_SECOND and SOS_HOLD. Together they let the firmware tell short touches, double taps, long presses and emergency holds apart, and reject transient electrostatic noise through software debouncing.

Table 4: Touch gestures and device response

Gesture	Device response
Short touch	Wakes the device; acknowledges an active reminder
Double touch	Playful irritated reaction on the OLED
Long press	Shows device status and network connection state
SOS hold	Starts the emergency countdown (SOS_COUNTDOWN)
Release during countdown	Cancels the SOS and returns to IDLE

4.4 BLE Communication Protocol and JSON Messaging

The ESP32 exposes a BLE GATT service with a Nordic UART-style communication profile. Messages between the smartphone app and the ESP32 are lightweight JSON payloads, summarised in Table 5.

Table 5: BLE JSON message types

Message type	Direction	Purpose
ADD_REMINDER	App → ESP32	Sends a new reminder (title, time, repeat flags)
DELETE_REMINDER	App → ESP32	Removes a stored reminder by ID
SYNC_TIME	App → ESP32	Synchronises device epoch time with the phone clock
REMINDER_TRIGGERED	ESP32 → App	Reports that a scheduled reminder fired locally
REMINDER_ACK	ESP32 → App	Confirms the user touched the device to acknowledge
SOS	ESP32 → App	Signals an emergency after the countdown completes
TEST_SOS	App → ESP32	Runs a diagnostic test of the buzzer and alert sequence

4.5 Emergency SOS Workflow

A deliberate long hold on the touch sensor moves the firmware into SOS_COUNTDOWN, which shows an on-screen countdown with buzzer warnings. If the user releases the sensor before the countdown ends, the sequence is aborted and the device returns to IDLE, which protects against accidental activation. If the countdown completes, the device enters SOS_SENT and transmits an SOS message to the paired smartphone, which executes the configured emergency communication workflow (Figure 4).

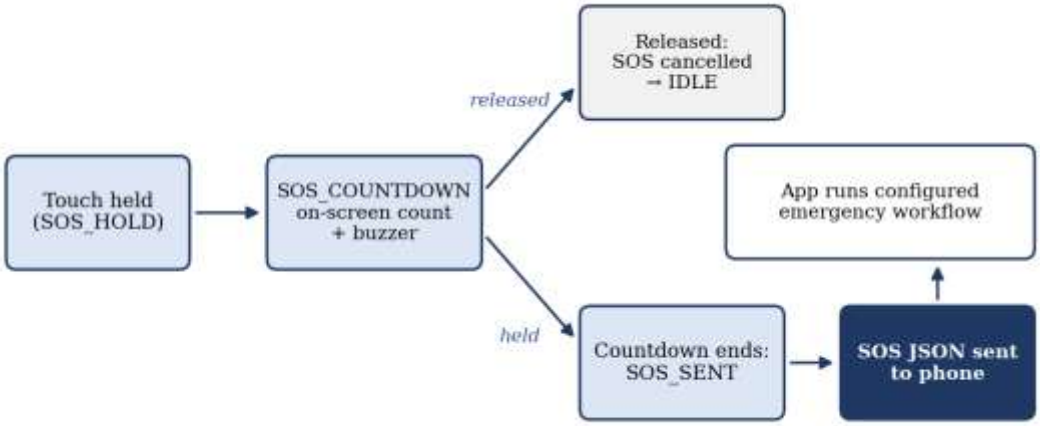


Figure 4: Emergency SOS workflow with cancel option

4.6 Data Flow and Reminder Workflow

- **Configuration:** the caregiver enters reminders and emergency contacts in the Android application.
- **Synchronisation:** data is sent over BLE and stored on the ESP32 in non-volatile storage (Preferences).
- **Execution:** the ESP32 internal clock monitors active schedules using non-blocking millis() timing.
- **Trigger:** when a scheduled time arrives, the device enters the REMINDER state, shows text on the OLED and pulses the buzzer.
- **Acknowledgment:** the user touches the TTP223 sensor; the ESP32 clears the alert, returns to IDLE and sends a REMINDER_ACK packet to the smartphone.

5. Results and System Behavior

5.1 Experimental Setup and Prototyping

The prototype was assembled on a breadboard with the ESP32 NodeMCU, the OLED over I2C, the TTP223 module and an active buzzer, powered from 5 V USB. Testing covered firmware stability, touch debounce, BLE reconnection and rendering.

5.2 Functional Testing and Verification

Twelve test cases evaluated hardware and software reliability (Table 6); all passed.

Table 6: Functional test cases

Test case	Expected result	Status
Touch detection	Clean recognition of a user touch via the TTP223	Passed
False short touch	Transient electrostatic noise rejected by software debounce	Passed
Double touch	Playful irritated reaction on the OLED	Passed
Long press	Displays device status and network connection state	Passed
SOS activation	Countdown starts when the touch sensor is held	Passed
SOS cancellation	Emergency sequence aborted if released during countdown	Passed
Reminder trigger	OLED reminder graphic and buzzer pattern activate	Passed
Reminder acknowledgment	Display clears on touch and the app is notified	Passed
BLE connection	Pairing and JSON packet exchange with the Android app	Passed
BLE disconnection	Graceful fallback to offline local operation without crashing	Passed
Sleep mode transition	Enters SLEEPING state with ‘Zzz’ after inactivity timeout	Passed
Wake-up behavior	Returns to IDLE immediately on a valid touch	Passed

5.3 Observed Prototype Behavior

- **Approachable companion interface:** the animated eyes provided a warm, engaging focal point. The automatic IDLE → SLEEPY → SLEEPING transition reduced unnecessary OLED illumination while keeping instant response to touch.
- **Reliable tactile interaction:** software debouncing filtered out environmental false triggers on the sensitive TTP223 module, so intentional touches produced reliable transitions.
- **Seamless offline-first execution:** because reminders are stored in the ESP32 non-volatile memory, scheduled alerts worked even when the smartphone was temporarily out of Bluetooth range.

5.4 Limitations of the Current Prototype

The evaluation so far is functional: it confirms that each behaviour works as designed on a breadboard prototype, but it does not include a field trial with elderly users, measurements of BLE range or battery life, or a long-duration reliability test. SOS delivery currently depends on the paired smartphone being nearby and connected (see Chapter 8).

6. Societal Relevance

CareBuddy addresses socio-economic and demographic challenges linked to ageing populations and independent living.

Enhanced independence: it helps elderly people keep to their medication and hydration schedules without constant direct supervision.

Caregiver peace of mind: it bridges the communication gap between elderly people and busy family members or caregivers through acknowledgment tracking and emergency alerts.

Affordable assistive technology: by using low-cost open-source hardware (ESP32 and commodity sensors), CareBuddy offers a cost-effective alternative to expensive commercial monitoring systems.

Human-centred design: it combines tactile, visual and auditory modalities to accommodate seniors with mild cognitive or sensory impairment, promoting dignity and comfort in ageing in place.

7. Conclusion

CareBuddy demonstrates the integration of embedded systems, Internet of Things (IoT) communication, mobile application development and human-centred design in an interactive elderly assistance prototype. By combining an ESP32 microcontroller, an SSD1306 OLED display, a TTP223 capacitive touch sensor, an active buzzer and BLE smartphone connectivity, the project delivers a reliable, low-cost solution for daily reminders and emergency safety.

Non-blocking firmware state machines, local non-volatile reminder storage and structured JSON messaging give robust, responsive behaviour, and all twelve functional test cases passed. CareBuddy shows that accessible assistive devices can be built from commodity hardware to improve the safety, independence and daily well-being of elderly users.

8. Future Scope

Cellular integration: add a low-cost GSM/LTE module (such as the SIM800L) for direct SMS and emergency calling independent of a smartphone bridge.

Location tracking: add a GPS module to support outdoor safety monitoring for early-stage dementia or wandering-risk individuals.

Cloud caregiver dashboard: extend the Android/web ecosystem so remote caregivers can monitor compliance analytics and reminder history over cloud servers.

Voice interaction: integrate offline voice recognition for simple spoken commands (for example ‘Confirm medicine’) to help visually impaired users.

Fall detection: add an inertial measurement unit such as the MPU6050 to detect sudden falls automatically and trigger the SOS protocol.

User trials and measurements: run field trials with elderly users and measure BLE range, battery life and long-duration reliability.

9. References

[1] Espressif Systems, "ESP32 Series Datasheet," 2026. Available: https://www.espressif.com/

[2] Arduino Foundation, "Arduino Reference Documentation," 2026. Available: https://www.arduino.cc/reference/en/

[3] Adafruit Industries, "Adafruit SSD1306 OLED Display Library Documentation," 2025. Available: https://learn.adafruit.com/

[4] Bluetooth Special Interest Group (SIG), "Core Specification Version 5.3," 2023. Available: https://www.bluetooth.com/

[5] Android Developers, "Kotlin and Jetpack Compose Documentation," 2026. Available: https://developer.android.com/

[6] TTP223 Manufacturer, "TTP223 Touch Pad Detector IC Datasheet," Giantec Semiconductor.